

CAPSTONE PROJECT CHARTER & TEAM SPECIFICATIONS

Student 1 (Lead / Full Name):

Student 2 (Partner Full Name):

Department & Year:

Official Project Title:

Primary Technology Stack:

Sprint Start Date:

RUNBOOK INSTRUCTIONS & CARE PROMPTING GUIDELINES

1. This Runbook is your official engineering portfolio record for Sathyabama Institute c
2. Execute across the 4 Disciplined SDLC Phases: Phase 1 (Define) -> Phase 2 (Design) -
3. Apply the CARE Formula for all AI prompts: C (Context) -> A (Action) -> R (Result) ->
4. In Phase 2, model 3-tier System Architecture and 3NF Relational Database Schemas
5. In Phase 3, build robust Spring Boot 3 / FastAPI REST APIs, MySQL JPA persistence, a
6. In Phase 4, resolve all CORS & HTTP issues, package a 1-click startup script, and deli
7. Replace all placeholder brackets '[FILL HERE]' with your team's actual code, diagrams

SIST CSE AI EVALUATION & VIVA RUBRIC (100 MARKS TOTAL)

SDLC Phase

Phase 1: DEFINE

Phase 2: DESIGN

Phase 3: DEVELOP

Phase 3: INTEGRATE

Phase 4: DEPLOY

TOTAL SCORE

TECHNOLOGY (SIST)

CSE AI | Capstone Runbook

Shudiksha Sugeerthanaa
Sofiya S
B.E. CSE (Artificial Intelligence) - Final Year
Smart Pharmacy Supply Chain System
Java Spring Boot 3 (REST API + Spring Data JPA), MySQL 8.0, HTML5, CSS3, Vanilla JS
Sep-26

Project Overview

of Science and Technology (SIST).

-> Phase 3 (Develop) -> Phase 4 (Deploy).

> E (Example/Clarification).

on Draw.io (<https://app.diagrams.net>).

and responsive HTML5/CSS Grid frontends.

ver a live 3-minute technical viva pitch.

s, API JSON, and active GitHub URLs.

Evaluation Criteria & Milestone Deliverable
Problem statement, user personas, Given/When/Then acceptance criteria
Draw.io system architecture diagram, 3NF MySQL schema, REST API contracts
Responsive Frontend UI (HTML5, CSS Grid), Spring Boot REST API & JPA
MySQL database persistence, JavaScript Fetch API & CORS resolution
Cloud deployment / 1-click startup script, GitHub documentation, 3-min viva pitch
Grand Aggregate Score across all Full-Stack Engineering Dimensions

Student 1 Register No:	44731123
Student 2 Register No:	44731112
Batch / Section:	2023 - 2027 / Section A1
Project Domain:	Hospital Pharmacy Inventory & Pharmaceutical Supply Chain Ma
Project Trainer / Mentor:	Department Faculty Guide / Training Mentor
Target Deployment & Viva Date:	September 2026 (Final Year Project Viva)



Max Marks	Evaluator Score
15	[]
20	[]
25	[]
20	[]
20	[]
100	[]

nagement



PHASE 1: DEFINE — PROBLEM STATEMENT

Dimension
1. Project Title
2. Real-World Problem
3. Target User Personas
4. Proposed Technical Solution
5. Core Value Metric (KPI)
6. Scope Exclusions (Out of Scope)

NT & PROJECT SCOPE

Guiding Engineering Question / Explanation
What is the official title of your full-stack web/AI application?
What painful manual problem does this project eliminate? Who is impacted?
Who are the primary end-users of this application? (List specific roles)
How does your full-stack application architecturally resolve the problem?
How will you quantify the technical success of this application?
What complex features are intentionally deferred from this MVP?

Student Team Project Response (Edit & Complete)

Smart Pharmacy Supply Chain System

Hospital pharmacies face critical clinical and financial operational vulnerabilities: (1) Drug wastage caused by expired medicines remaining undetected on pharmacy shelves; (2) Counterfeit or suspect medicine batches infiltrating dispensing channels; (3) Improper dispensing order (absence of FEFO enforcement) causing avoidable expiration write-offs; (4) Lack of monitoring for

1. Primary User: Pharmacy Staff / Dispensing Pharmacist (Registers medicines and batches, executes FEFO dispensing, verifies batch authenticity, records cold-chain temperature logs).

2. Pharmacy Administrator / Store Manager (Registers certified suppliers, monitors low-stock alerts, tracks multi-tier expiry radar, audits suspect/rejected batches)

A layered 3-tier full-stack architecture:

1. Presentation Tier: Responsive single-portal frontend (HTML5, CSS Grid/Flexbox, Vanilla JS Fetch API) across 8 specialized functional modules.

2. Application Tier: Java Spring Boot 3 RESTful API enforcing FEFO (First-Expired-First-Out) multi-100% strict adherence to FEFO dispensing order, 0% dispensing of expired or rejected batches, 100% automated detection and flagging of cold-chain temperature excursions, sub-second API response time (<100ms locally), and 0 double-decrement stock calculation errors.

Intentionally excluded to preserve academic clarity and eliminate external failure modes: JWT authentication/login, payment gateway integration, blockchain, AI/ML black-box models, OCR, physical IoT sensor hardware, Docker/Microservices, external cloud third-party APIs, and SMS/Email external gateways

PHASE 1: DEFINE — USER STORIES & ACCEPTANCE

Story ID	User Persona (As a...)
US-01	Pharmacy Staff / Pharmacist
US-02	Pharmacy Staff / Pharmacist
US-03	Pharmacy Administrator / Store Manager
US-04	Pharmacy Staff / Auditor
US-05	Pharmacy Administrator

CRITERIA

Action / Feature (I want to...)
Dispense medicine by selecting medicine ID and specifying quantity
Authenticate a medicine batch by scanning or searching its unique batch identification number
Monitor an automated multi-tier Expiry Radar with real-time categorizations
Log periodic temperature sensor readings for cold-chain medicine batches
View an executive dashboard with 5 KPI metric cards and 3 real-time alert tables

Business Benefit (So that...)
The system automatically selects the earliest-expiring verified batch (FEFO) and deducts stock, preventing medication expiry losses.
I can verify authenticity, view manufacturer provenance, and transition the status to VERIFIED or REJECTED before dispensing.
I can proactively identify batches nearing expiration and quarantine expired stock before clinical dispensing.
The system automatically checks readings against the medicine's allowed temperature thresholds and flags excursions.
I have instant single-pane visibility over total medicines, active batches, low-stock items, expiring stock, and cold-chain violations.

Acceptance Criteria (Given / When / Then)	Priority
1. Given verified active batches exist with remaining stock, When staff dispenses medicine, Then system selects the earliest expiring verified batch.	P0 (Must Have)
1. Given a batch number is entered, when verification is requested, Then system returns status: VERIFIED, UNVERIFIED, REJECTED, or NOT FOUND.	P0 (Must Have)
1. Automatically computes days remaining until expiry across all active stock. 2. Classifies batches into 4 distinct tiers: Expired (<0 days),	P0 (Must Have)
1. Given a temperature reading is entered for a batch, 2. When submitted, Then system compares reading with medicine minTemperature and maxTemperature.	P1 (High)
1. Dashboard displays 5 summary cards: total medicines, Active Batches, Low Stock (<=20 units), Expiring Soon (<=90 days), and Expired. 2. Renders 2 recent audit tables: Recent Expiry Alerts	P1 (High)



PHASE 2: DESIGN — SYSTEM ARCHITECTURE

Architectural Layer
Step 1: Access Modeling Tool
Step 2: Tier 1 — Client Layer
Step 3: Network API Contract
Step 4: Tier 2 — Backend Engine
Step 5: Database Connection
Step 6: Tier 3 — Relational DB
Step 7: Diagram Embed / Link

: & DRAW.IO MODELING

Draw.io Modeling Instructions & Guidelines

Open <https://app.diagrams.net> (Free Draw.io). Create a blank diagram and select Software Architectu

Draw a 'Browser' container representing the Client Frontend.Specify: Origin Port 5500, HTML5 UI, CSS

Draw a bidirectional arrow between Client and Backend Server.Label: 'REST API over HTTP/1.1 (JSON

Draw a 'Web Server' box representing Java Spring Boot 3 / Python FastAPI.Show 3 internal layers: Con

Draw an arrow from Backend Service to Database.Label: 'JDBC Connection Pool (TCP/IP Port 3306) /

Draw a 'Database Cylinder' shape representing MySQL 8.0 on disk.Label: Target schema and tables (e.

Export your Draw.io diagram as high-resolution PNG or shareable link and paste here:

Student System Specifications / Diagram Link

Draw.io Web App (Exported as High-Resolution PNG & Embedded in README.md)

Tier 1: Responsive Single-Portal Client Web Application (HTML5, CSS3 Grid/Flexbox, Vanilla JS Fetch API across 8 pages: index.html, medicines.html, suppliers.html, batches.html, dispense.html, expiry.html, temperature.html, verification.html)

RESTful API over HTTP/1.1 (Base URL: http://localhost:8080/api, JSON Payloads, CORS Filter enabled for all origins, Unified ApiResponse Envelope)

Tier 2: Java 17 + Spring Boot 3 Layered Architecture (@RestController -> @Service with @Transactional -> @Repository with Spring Data JPA + GlobalExceptionHandler)

HikariCP JDBC Connection Pool (Port 3306), Hibernate ORM 6.4 with MySQLDialect / H2Dialect fallback, DDL Auto Update

Tier 3: MySQL 8.0 Relational Database (pharmacy_db schema normalized in 3NF: medicines, suppliers, batches, temperature_logs, dispense_records, dispense_items)

Mermaid & Draw.io Architectural Diagram available in project README.md under Section 5 (System Architecture) and Section 6 (DFD Level 0).

PHASE 2: DESIGN — MYSQL RELATIONAL DAT

Table Name	Column Name
medicines	id
medicines	name
medicines	generic_name
medicines	category
medicines	dosage
medicines	min_temperature
medicines	max_temperature
medicines	created_at
suppliers	id

suppliers	supplier_name
suppliers	contact_person
suppliers	phone
suppliers	email
suppliers	address
batches	id
batches	batch_number
batches	medicine_id
batches	supplier_id
batches	manufacturer
batches	manufacturing_date
batches	expiry_date
batches	quantity
batches	storage_requirement
batches	verification_status
temperature_logs	id
temperature_logs	batch_id
temperature_logs	recorded_temperature
temperature_logs	recorded_at
temperature_logs	compliance_status
temperature_logs	notes
dispense_records	id
dispense_records	medicine_id
dispense_records	total_requested_quantity
dispense_records	total_dispensed_quantity
dispense_records	dispensed_to
dispense_records	dispensed_at
dispense_items	id
dispense_items	dispense_record_id

dispense_items
dispense_items
dispense_items
dispense_items
dispense_items

batch_id
batch_number
expiry_date
quantity_dispensed
remaining_batch_stoc

'ABASE SCHEMA (3NF)

Data Type & Length	Key Constraint	Nullability / Default
BIGINT AUTO_INCREMENT	PRIMARY KEY	NOT NULL
VARCHAR(100)	UNIQUE	NOT NULL
VARCHAR(100)	NONE	NOT NULL
VARCHAR(50)	NONE	NOT NULL
VARCHAR(50)	NONE	NOT NULL
DOUBLE	NONE	NOT NULL
DOUBLE	NONE	NOT NULL
DATETIME	NONE	DEFAULT CURRENT_TIMESTAMP
BIGINT AUTO_INCREMENT	PRIMARY KEY	NOT NULL
VARCHAR(100)	UNIQUE	NOT NULL
VARCHAR(100)	NONE	NOT NULL
VARCHAR(20)	NONE	NOT NULL
VARCHAR(100)	NONE	NOT NULL
VARCHAR(255)	NONE	NOT NULL
BIGINT AUTO_INCREMENT	PRIMARY KEY	NOT NULL
VARCHAR(50)	UNIQUE	NOT NULL
BIGINT	FOREIGN KEY	NOT NULL
BIGINT	FOREIGN KEY	NOT NULL
VARCHAR(100)	NONE	NOT NULL
DATE	NONE	NOT NULL
DATE	NONE	NOT NULL
INT	NONE	NOT NULL DEFAULT 0
VARCHAR(100)	NONE	NOT NULL
VARCHAR(20)	NONE	DEFAULT 'UNVERIFIED'
BIGINT AUTO_INCREMENT	PRIMARY KEY	NOT NULL
BIGINT	FOREIGN KEY	NOT NULL
DOUBLE	NONE	NOT NULL
DATETIME	NONE	DEFAULT CURRENT_TIMESTAMP
VARCHAR(20)	NONE	NOT NULL
VARCHAR(255)	NONE	NULL
BIGINT AUTO_INCREMENT	PRIMARY KEY	NOT NULL
BIGINT	FOREIGN KEY	NOT NULL
INT	NONE	NOT NULL
INT	NONE	NOT NULL
VARCHAR(100)	NONE	NULL
DATETIME	NONE	DEFAULT CURRENT_TIMESTAMP
BIGINT AUTO_INCREMENT	PRIMARY KEY	NOT NULL
BIGINT	FOREIGN KEY	NOT NULL

BIGINT	FOREIGN KEY	NOT NULL
VARCHAR(50)	NONE	NOT NULL
DATE	NONE	NOT NULL
INT	NONE	NOT NULL
INT	NONE	NOT NULL

Description & Business Purpose
Unique surrogate key for pharmaceutical drug catalog item
Brand name of the drug (e.g., Paracetamol, Insulin Glargine)
Active pharmaceutical ingredient (API) generic chemical name
Therapeutic classification (e.g. Analgesic, Antibiotics, Antidiabetic)
Formulation strength / unit dosage (e.g. 500mg, 100 IU/ml, 1g Vial)
Minimum permissible storage temperature in Celsius (e.g. 2.0 for Cold Chain)
Maximum permissible storage temperature in Celsius (e.g. 8.0 for Cold Chain)
Drug registration audit timestamp
Unique surrogate key for certified distributor/supplier

Corporate name of pharmaceutical manufacturer or distributor

Designated commercial point of contact

Official phone contact number

Official procurement email address

Physical warehouse / corporate facility address

Unique surrogate primary key for physical inventory batch

Unique manufacturer batch identification code (INDEXED)

References medicines(id) ON DELETE RESTRICT

References suppliers(id) ON DELETE RESTRICT

Physical manufacturing laboratory facility

Production date (must be strictly before expiry_date)

Expiration date used by FEFO sorting query (INDEXED)

Physical unit count remaining in pharmacy stock

Storage category (e.g. Cold Storage 2-8°C, Room Temp 15-25°C)

Anti-counterfeit state: UNVERIFIED, VERIFIED, or REJECTED (INDEXED)

Unique surrogate key for cold-chain audit event

References batches(id) ON DELETE CASCADE

Actual thermal sensor reading in Celsius

Timestamp of thermal logging event

Evaluated compliance status: COMPLIANT or OUT_OF_RANGE

Clinical diagnostic or sensor location notes

Unique surrogate key for dispensing transaction header

References medicines(id) ON DELETE RESTRICT

Requested dispensing quantity

Actual total units fulfilled via FEFO

Clinical department, ward, or patient identifier

Dispensing transaction timestamp

Unique line item surrogate key for FEFO batch deduction

References dispense_records(id) ON DELETE CASCADE

References batches(id) ON DELETE RESTRICT
Snapshot of deducted batch number
Snapshot of deducted batch expiry date
Units deducted from this specific batch
Remaining stock balance in batch after deduction



PHASE 2: DESIGN — RESTFUL API CONTRAC

API ID	HTTP Verb
API-01	POST
API-02	GET
API-03	POST
API-04	GET
API-05	POST
API-06	GET
API-07	POST
API-08	POST

API TESTS & POSTMAN SPECS

Endpoint Route	Request Body Payload (JSON)	Expected Status
/api/dispense	{"medicineId": 1, "quantity": 10, "dispensedTo": "Ward 4A", "notes": "FEFO Test"}	200 OK
/api/batches/medicine/{id}/available	None (Medicine ID in Path)	200 OK
/api/batches/verify	{"batchNumber": "BATCH001", "action": "VERIFY"}	200 OK
/api/batches/expiry-alerts?category=CRITICAL_30	None (Optional query parameter)	200 OK
/api/temperature-logs	{"batchId": 6, "recordedTemperature": 5.0, "notes": "Routine audit"}	201 Created
/api/dashboard/stats	None (Empty Body)	200 OK
/api/medicines	{"name": "Paracetamol", "genericName": "Paracetamol"}	201 Created
/api/batches	{"batchNumber": "BATCH-PCM-001", "medicineId": 1, "expiryDate": "2024-12-31", "notes": "Initial stock check"}	201 Created

Success Response JSON
<pre>{"status": "SUCCESS", "data": {"recordId": 1, "medicineName": "Paracetamol", "totalDispensedQuantity": 10, "dispensedBatches": [{"batchNumber": "BATCH-PCM-001",</pre>
<pre> {"status": "SUCCESS", "data": [{"id": 1, "batchNumber": "BATCH-PCM-001", "expiryDate": "2026-10-10", "quantity": 20}]}</pre>
<pre> {"status": "SUCCESS", "data": {"batchNumber": "BATCH001", "status": "VERIFIED", "exists": true, "medicineName": "Amoxicillin"}}</pre>
<pre> {"status": "SUCCESS", "data": [{"batchNumber": "BATCH- CRT-030", "daysRemaining": 17, "alertCategory": "CRITICAL_30"}]}</pre>
<pre> {"status": "SUCCESS", "data": {"id": 5, "recordedTemperature": 5.0, "complianceStatus": "COMPLIANT"}}</pre>
<pre> {"status": "SUCCESS", "data": {"totalMedicines": 5, "activ {"status": "SUCCESS", "data": {"id": 1, "name": "Paraceta {"status": "SUCCESS", "data": {"id": 1, "batchNumber": "I</pre>

Error Handling Spec
409 Conflict (Insufficient stock / no verified batches) / 400 Bad Request
404 Not Found (Medicine ID does not exist)
Returns status: NOT FOUND with exists: false if batch number does not exist
500 Internal Server Error
404 Not Found (Batch not found) / 400 Bad Request

500 Internal Server Error

409 Conflict (Duplicate medicine name) / 400 Bad Request (minTemp > maxTemp)

409 Conflict (Duplicate batch #) / 400 Bad Request (mfgDate >= expDate)



PHASE 3: DEVELOP — CARE AI PROMPT ENGINEER

Log ID	AI Tool Used
AI-01	Antigravity (Gemini 3.8 Flash)
AI-02	Antigravity (Gemini 3.8 Flash)
AI-03	Antigravity (Gemini 3.8 Flash)
AI-04	Antigravity (Gemini 3.8 Flash)

ING LOG

SDLC Task	CARE Prompt Sent to AI
Generate FEFO Dispensing Service Engine	Context: Final-year B.E. CSE AI Smart Pharmacy Supply Chain System. Action: Write Spring Boot service method dispenseMedicine(DispenseRequest). Strictly sort verified context: cold-chain monitoring for temperature-sensitive drugs like Insulin (2°C - 8°C).
Generate Cold-Chain Temperature Compliance Evaluator	Action: Write Spring Boot service recordTemperature(TemperatureLogRequest). Compare context: Smart pharmacy web portal with Temperature modules.
Design Vanilla CSS3 Design System & UI Architecture	Action: Create modern, responsive Vanilla CSS stylesheet using clinical teal and slate color palette, CSS Grid/Flexbox, context: REST API for handling for college project viva.
Global Exception Handling & Bean Validation	Action: Create @RestControllerAdvice mapping ResourceNotFoundException (404), DuplicateResourceException (409), InsufficientStockException (400) and

Code Output Received

Generated DispenseServiceImpl.java implementing findAvailableBatchesForFebo, quantity split deductions, and DispenseRecord/DispenseItem persistence.

Generated TemperatureLogServiceImpl.java with automatic threshold evaluation and diagnostic excursion note generation.

Generated frontend/css/styles.css and 8 semantic HTML5 pages (index, medicines, suppliers, batches, dispense, expiry, temperature, verification) with api.js Fetch client.

Generated GlobalExceptionHandler.java with explicit HTTP status code mappings matching project specification.

Student Review & Verification Notes

Reviewed and tested live via PowerShell. Verified 10 units deducted from Batch A; tested 25 units split (10 from Batch A, 15 from Batch B); tested 1000 units rejected with 409 Conflict.

Verified live via REST API. 5.0°C returned COMPLIANT; 12.0°C returned OUT_OF_RANGE; 1.0°C freeze breach returned OUT_OF_RANGE.

Verified responsive layout across desktop and mobile; verified zero CDN dependencies; verified live data binding.

Tested edge cases: duplicate batch returned 409; invalid date order returned 400; insufficient stock returned 409.

PHASE 3: DEVELOP — FRONTEND

UI Component
1. Executive Dashboard
2. Medicine Catalog Management
3. Certified Supplier Directory
4. Batch Inventory Ledger
5. FEFO Dispensing Console

- 6. Multi-Tier Expiry Radar
- 7. Cold-Chain Temperature Monitor
- 8. Anti-Counterfeit Batch Scanner

UI IMPLEMENTATION (HTML5 & CSS3)

Key HTML5 / CSS3 Techniques Used	Source File Path
5 KPI cards (Total Medicines, Active Batches, Low Stock, Expiring Soon, Expired), 3 alert tables	frontend/index.html
CSS Grid table, Add/Edit modal, min/max temperature controls, instant search filter	frontend/medicines.html
Responsive table, Add/Edit modal, contact & email links	frontend/suppliers.html
Filterable table, status badges, Add Batch modal, date pickers	frontend/batches.html
Live available batch queue, quantity calculator, visual receipt breakdown card	frontend/dispense.html

Multi-tab filter (All, Expired, <=30d Critical, <=60d Warni frontend/expiry.html

Audit simulator, threshold check badges, historical log t frontend/temperature.html

Batch lookup bar, dynamic status card (VERIFIED, UNVEI frontend/verification.html

Implementation Snippet / Technical Notes	Status
Single-pane pharmacy overview dashboard displaying live aggregated counts and recent alerts without heavy chart libraries.	Completed
Complete drug catalog management with client-side search and cold-chain temperature threshold specification.	Completed
Maintains authorized manufacturer and vendor credentials for pharmaceutical supply chain traceability.	Completed
Physical inventory ledger tracking unique batch numbers, expiry dates, quantities, and verification states.	Completed
Core academic console enforcing First-Expired-First-Out dispensing with itemized batch deduction receipt.	Completed

Automated expiry radar classifying inventory into urgent

Completed

Validates actual thermal sensor readings against drug specifications

Completed

Enables pharmacy staff to verify batch authenticity, inspect

Completed

PHASE 3: DEVELOP — BACKEND & MYSQL PERSISTENCE

Architecture Layer	Source File / Class
1. Configuration & Seeder	application.properties, CorsConfig.java,
2. JPA Entities (3NF)	DentalOffice.java, Supplier.java, Batch.java,
3. Data Repositories	TreasureRepository.java, SupplierRepository.java,
4. Business Services	DispenseServiceImpl.java, BatchServiceImpl.java,
5. REST Controllers	DispenseController.java, BatchController.java, MedicineController.java
6. Exception Handling	GlobalExceptionHandler.java

ENCE (SPRING BOOT)

Annotations Used
@Configuration, @Bean, @Component, CommandLineRunner
@Entity, @Table, @Id, @GeneratedValue, @ManyToOne, @OneToMany, @Column(unique=true)
@Repository, JpaRepository, @Query (JPQL)
@Service, @Transactional
@RestController, @RequestMapping, @CrossOrigin, @Valid
@RestControllerAdvice, @ExceptionHandler

Business Logic & Responsibilities Implemented	Verification Result
Configured MySQL 8.0/5.7 and TZ fallback profile, CORS filter allowing all local origins; CommandLineRunner auto-seeding realistic medicines, suppliers, batches, and sold	Connected to MySQL & Seeded
Normalized 3NF relational schema with unique batch index, date validation, and cascade audit relationships.	Tables auto-generated
Extended Spring Data Jpa repository with custom JPQL query findAvailableBatchesForFifo ordering verified	Zero boilerplate SQL
core academic business logic: ABC multi-batch inventory fulfillment, multi-tier expiry calculations (90d, 60d, 30d, exposed rest temperature, api dispense, checking	All business tests passed
/api/batches/verify, /api/expiry-alerts, /api/temperature-logs, /api/dashboard/stats) returning unified ApiResponse	Verified via cURL & Postman
Centralized REST error handling: maps ResourceNotFoundException Verified status codes	

 PHASE 3: DEVELOP — FULL-STACK V

Integration Milestone
1. Intercept Form Submissions
2. Extract & Validate Form Inputs
3. JSON Payload Serialization
4. Asynchronous HTTP Dispatch
5. Unified Response Parsing & DOM Update

MIRING & FETCH API INTEGRAT

Technology / API Used
event.preventDefault() in api.js
DOM extraction + Validation
JSON.stringify(payload)
fetch(API_BASE_URL + endpoint, options)
await response.json() + DOM rendering

ION

Technical Implementation Details
Intercepted submit events on all 8 forms (Dispense, Batch, Medicine, Temp, Verification) preventing full page reloads.
Extracted field values; verified numeric ranges, positive quantities, and mfgDate < expDate constraint before dispatch.
Serialized JavaScript objects into standardized JSON payloads with Content-Type: application/json header.
Dispatched async HTTP requests to http://localhost:8080/api endpoints (GET, POST, PUT, DELETE, PATCH).
Parsed unified ApiResponse envelopes; rendered dynamic FEFO receipt cards, updated tables, and triggered toast alerts.

Browser Outcome & Verification
Smooth asynchronous operations
Client-side validation passed
Clean REST JSON payload
Network calls logged with 200/201/409
UI updates instantly without page flash

 PHASE 4: DEPLOY — SYSTEMAT

Bug ID
BUG-01
BUG-02
BUG-03
BUG-04

BUG-05

IC BUG DIAGNOSIS & RESOLUTION LOG

Symptom Observed / Error Code	Diagnostic Tool Used
CORS policy blocked browser request on fetch calls.	Chrome DevTools Console
Backend returned 400 Bad Request when creating Batch with raw entity.	Postman / DevTools Network Tab
Dispense request exceeding available stock failed silently.	Service Unit Testing
Duplicate batch number insert crashed with unhandled 500 SQL error.	MySQL Terminal / Stack Trace

Valid boundary temperature readings (e.g. 2.0°C for 2-4°C) Cold-Chain Scenario Test

Root Cause Identified
Origin http://localhost:5500 / file origin not authorized by backend port 8080.
Batch entity had @NotNull on Medicine and Supplier objects which were null before controller ID mapping.
Service did not validate total available stock before initiating loop, resulting in partial deduction.
Database unique constraint on batch_number threw DataIntegrityViolationException.
Service used strict inequality (< and >) instead of inclusiv

Code Fix Applied	Resolved By
Configured CorsConfig.java bean with allowed origin patterns '*' and added @CrossOrigin(origins = '*') on all controllers	Student Team
Created BatchRequestDTO.java separating HTTP payload (medicineId, supplierId) from JPA entity relationships.	Student Team
Added total stock validation in DispenseServiceImpl and created InsufficientStockException mapped to HTTP 409 Conflict	Student Team
Added batchRepository.existsByBatchNumber(...) check in service throwing DuplicateResourceException mapped to HTTP 409 Conflict	Student Team
Updated comparison logic to: actualTemp >= minTemp { Student Team	

PHASE 4: DEPLOY — CLOUD DEPLOYMENT & CI/CD

Deployment Deliverable
Public GitHub Repository
Live Frontend Web Application
Live Backend REST API
Recruiter README.md File

 3-MINUTE TECHNICAL VIVA PITCH SCRIPT FOR RECRUITERS
Minute 1: The Problem & Value
Minute 2: System Architecture & Tech Stack
Minute 3: Live Demo & Engineering Robustness

3-MINUTE PITCH SCRIPT

Target Platform
Local Git / GitHub
Local Web Portal / Static Server
Spring Boot 3 REST Server
Academic Project Report

Good morning/afternoon respected examiners! I built the Smart Pharmacy Supply Chain to address the clinical hazard of counterfeit drug batches, and loss of potency in cold-chain drugs like vaccines, and ensure compliance and zero expired drug issuance.

Architecturally, Tier 1 is a responsive Vanilla HTML5/CSS3/JavaScript single-portal from Java 17 Spring Boot 3 backend implementing a layered architecture (@RestController, 404, 409). Tier 3 is MySQL 8.0 normalized in Third Normal Form (3NF) maintaining ACI

In this live demo, notice how requesting 10 units of Paracetamol automatically selects seamlessly splits the deduction across Batch A and Batch B. If an unverified or expired COMPLIANT, while 12.0°C immediately triggers an OUT_OF_RANGE alert. We resolve:

Live URL / Repository Link (Fill by Student)
https://github.com/shudiksha-m/smart-pharmacy-supply-chain
file:///c:/Users/Shudiksha M/Desktop/project/frontend/index.html
http://localhost:8080/api/dashboard/stats
Complete README.md with DFD Level 0, Architecture, 3NF ERD, API Specs, Viva Q&A



in System to solve severe pharmaceutical inventory challenges: multi-million dollar me
 e Insulin. Our application provides an automated, responsive 3-tier solution that guaran

itend across 8 modules consuming REST APIs via asynchronous Fetch API without heavy
 @Service with @Transactional, and Spring Data JPA) with a Global Exception Handler r
 D transaction integrity.

Batch A because its expiry date is earlier, executing FEFO allocation. When requesting 2
 batch exists, the system strictly locks it out. On the Cold-Chain Console, entering 5.0Â°C
 d CORS and handled all edge cases like duplicate batches and stock shortages with HT

Verification Status
Verified Active
Live in Browser
Responds 200 OK
Complete



dication expiry losses, the
tees 100% FEFO dispensing

y framework bloat. Tier 2 is a
napping status codes (400,

:5 units, the engine
C for Insulin (2-8Â°C) tags
TP 409 Conflict.